COEP TECHNOLOGICAL UNIVERSITY

MINI-PROJECT REPORT

LOAN APPROVAL PREDICTION USING MACHINE LEARNING, CLASS-IMBALANCE HANDLING AND FEATURE ENGINEERING

Submitted by

Rohit Selokar

MIS No.: 712652053
M.Tech – Data Science

Guided by

Dr. Yashodhara Haribhakta

Department of Computer Science and Engineering
COEP Technological University
2026–2027

ABSTRACT

Banks and other lenders receive large numbers of loan applications and must decide quickly and consistently which of them to approve. This project builds and evaluates machine learning models for this decision using the Kaggle Playground Series S4E10 loan dataset, which contains 58,645 applications described by 11 applicant, loan and credit-history attributes. Only 14.24% of the applications belong to the positive class (loan_status = 1, labelled Approved in the dataset description), so the data is strongly imbalanced (about 6 : 1) and accuracy alone is a misleading measure of quality.

After removing the identifier and converting three impossible values (ages and employment lengths of 123 years) to missing values, six features were derived from the original columns: employment_stability_ratio, credit_maturity_ratio, loan_to_income_burden, income_per_employment_year, credit_history_coverage and an experimental 0–100 Credit Profile Score built from income, loan affordability, employment length, credit-history length and previous default. The data was split in a stratified 70/15/15 ratio into training (41,051), validation (8,797) and test (8,797) sets. Class imbalance was handled with SMOTE, applied to the training data only inside a leakage-safe pipeline, which raised the share of the positive class in the training set from 14.2% to 50%.

Logistic Regression, Random Forest and Gradient Boosting (HistGradientBoostingClassifier) were compared on the validation set. Gradient Boosting was selected because it had the best F1-score (0.7991) and ROC-AUC (0.9519). On the untouched test set it reached an accuracy of 0.9525, precision of 0.9179, recall of 0.7318, F1-score of 0.8144 and ROC-AUC of 0.9531, correctly identifying 917 of the 1,253 positive applications while wrongly flagging 82 negative ones. Adding the six derived features changed validation F1-score by only +0.004 (Logistic Regression), +0.004 (Random Forest) and −0.005 (Gradient Boosting), so the original features already carry most of the signal. Permutation importance shows that loan_percent_income, person_home_ownership, loan_intent, loan_grade and loan_int_rate drive the predictions.

The report also explains how the Credit Profile Score is calculated, shows worked examples, and analyses its relationship with the target. Limitations are discussed openly, including the synthetic nature of the data, the need to confirm the meaning of the target label, the fixed 0.5 threshold and a recall of 73%.

Keywords: *loan approval prediction, class imbalance, SMOTE, feature engineering, credit profile score, Logistic Regression, Random Forest, Gradient Boosting, permutation importance*

TABLE OF CONTENTS

Section	Page
Abstract	2
List of Figures	3
List of Tables	3
Chapter 1 – Introduction	4
Chapter 2 – Problem Statement and Objectives	6
Chapter 3 – Literature Review	7
Chapter 4 – Methodology	9
Chapter 5 – Results and Discussion	15
Chapter 6 – Conclusion and Future Scope	25
References	26

LIST OF FIGURES

Figure	Title
Figure 4.1	Overall system architecture
Figure 4.2	Credit Profile Score: maximum points per component
Figure 4.3	End-to-end methodology workflow
Figure 5.1	Class distribution in the original dataset
Figure 5.2	Distributions of the numerical input attributes
Figure 5.3	Share of class 1 by categorical input attributes
Figure 5.4	Spearman correlation heatmap (original numeric features)
Figure 5.5	Training set before and after SMOTE
Figure 5.6	Validation model comparison
Figure 5.7	Original features versus original + six derived features
Figure 5.8	Credit Profile Score distribution by class
Figure 5.9	Share of class 1 by Credit Profile Score band
Figure 5.10	Permutation feature importance (Gradient Boosting)
Figure 5.11	Test confusion matrix and ROC curves

LIST OF TABLES

Table	Title
Table 3.1	Summary of relevant literature
Table 4.1	Dataset attributes
Table 4.2	Data preprocessing summary
Table 4.3	Derived (engineered) features
Table 4.4	Credit Profile Score components and formulas
Table 4.5	Stratified data split
Table 4.6	Model configurations
Table 4.7	Evaluation metrics
Table 5.1	Summary statistics of numerical attributes
Table 5.2	Validation model comparison
Table 5.3	Original versus derived features (validation)
Table 5.4	Credit Profile Score statistics by class
Table 5.5	Permutation feature importance
Table 5.6	Final test performance
Table 5.7	Test classification report

CHAPTER 1

INTRODUCTION

1.1 Background

Lending institutions receive thousands of loan applications. Each application contains information about the applicant (such as age, income and employment history), the requested loan (amount, purpose, interest rate and grade) and the applicant's credit history. A decision to approve or reject the application has to be made quickly, and it has to be consistent between applicants with similar profiles.

Manual screening is slow and can be subjective. Machine learning offers a data-driven alternative: a model learns patterns from past applications and produces a probability for each new application. Such a model can support, but not replace, the human decision maker by ranking applications and highlighting those that need attention.

Loan data has a practical difficulty that affects model building. The two outcomes are rarely equally frequent. In the dataset used in this project only about one application in seven belongs to the positive class. A model trained naively on such data can reach high accuracy simply by predicting the majority class, and for this reason imbalance handling and metric selection are central parts of this work.

1.2 Project Overview

The project, titled "Loan Approval Prediction Using Machine Learning, Class-Imbalance Handling and Feature Engineering", uses the Kaggle Playground Series S4E10 loan dataset. It carries out the following tasks:

- **Prediction:** predict the target loan_status (0 = Rejected, 1 = Approved, as defined in the dataset description) from applicant, loan and credit-history attributes.
- **Class-imbalance handling:** apply SMOTE to the training data only and measure its effect on the class balance.
- **Feature engineering:** derive six new features, including an interpretable 0–100 Credit Profile Score, and test whether they improve the models.
- **Model comparison:** compare Logistic Regression, Random Forest and Gradient Boosting using accuracy, precision, recall, F1-score and ROC-AUC.
- **Explanation:** use permutation importance to show which features the final model relies on.

1.3 Motivation

A lender needs to know not only whether a model is accurate, but also where it makes mistakes. In a 6 : 1 imbalanced problem a model that always predicts the majority class is about 85.8% accurate and yet identifies no positive application at all. The project is therefore motivated by the need for (i) a fair treatment of the minority class, (ii) metrics that expose both kinds of error, and (iii) a transparent, leakage-free pipeline that can be explained step by step.

A second motivation is to test a common belief in applied machine learning, namely that hand-crafted features improve results. Six features were designed from domain reasoning, and the project reports honestly how much they actually help.

1.4 Objectives

- Build machine learning models to predict loan approval (loan_status).
- Handle class imbalance using SMOTE without leaking information into the validation and test sets.

- Derive six new features, explain how each is calculated, and test their value.
- Explain in detail how the Credit Profile Score is calculated and how it relates to the target.
- Compare Logistic Regression, Random Forest and Gradient Boosting on a validation set.
- Evaluate the selected model once on an untouched test set using accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix.
- Identify the most influential features using permutation importance.

1.5 Scope

The work is based on one public dataset of 58,645 loan applications. The target is the dataset's loan_status column. The Credit Profile Score is an experimental score designed for this project; it is not an industry credit score. The study compares three model families with fixed settings and a fixed decision threshold of 0.5; a systematic hyperparameter search and a business-cost analysis of thresholds are outside the scope and are listed as future work.

1.6 Applications

- Supporting loan officers by ranking and pre-screening applications.
- Giving consistent first-level decisions for high-volume, low-value loans.
- Identifying which application attributes most influence a model's output.
- Providing a template for handling class imbalance in other binary decision problems.

1.7 Organisation of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 formalises the problem. Chapter 3 reviews the relevant concepts and methods. Chapter 4 describes the dataset, preprocessing, derived features (including the calculation of the Credit Profile Score), SMOTE, models and evaluation design. Chapter 5 presents the experimental results and discussion, including the limitations. Chapter 6 concludes the work and lists future improvements.

CHAPTER 2

PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Definition

Banks receive thousands of loan applications and must decide quickly and consistently which ones to approve. Manual screening is slow and subjective. The problem addressed in this project is to build a reliable, leakage-free machine learning model that predicts the loan decision recorded in the dataset, given the attributes of the application.

Formally, this is a binary classification problem. For an application i with attribute vector x(i), the model must predict $y(i) \in \{0, 1\}$, where 0 denotes Rejected and 1 denotes Approved according to the dataset description.

2.2 Formal Prediction Objective

Let X denote the feature vector of an application after preprocessing. The model estimates $P(Y = 1 \mid X)$. The predicted class is 1 when this probability is at least 0.5 and 0 otherwise. The threshold is kept at the default value of 0.5 for all models, so that the comparison between models is not affected by threshold tuning.

2.3 Class-Imbalance Formulation

The training data contains far fewer positive than negative examples (14.24% against 85.76% in the full dataset). Without correction, a learning algorithm can reduce its training error by favouring the majority class. The project therefore applies SMOTE, which creates synthetic minority examples, and evaluates the models on validation and test sets that keep the real class proportions.

2.4 Challenges

- Class imbalance (about 6 : 1) makes accuracy an incomplete measure of performance.
- Resampling must be applied to training data only; applying it before the split would leak synthetic information into the evaluation sets.
- Some records contain impossible values (123 years of age or employment) that must be handled without discarding the application.
- Derived features may carry no new information beyond the original columns, and this has to be measured rather than assumed.
- The dataset is synthetic, so conclusions may not transfer directly to a real lender.
- The meaning of the target label should be confirmed against the data source before any real-world use (see Section 5.10).

2.5 Expected Outcome

The expected outcome is a reproducible pipeline that takes a raw application, applies imputation, scaling and encoding, and returns the probability of the positive class. Alongside the pipeline the project delivers a validation-based model comparison, an untouched test evaluation, a feature-importance analysis and a documented Credit Profile Score. It is intended as a decision-support prototype rather than a production lending system.

CHAPTER 3

LITERATURE REVIEW

3.1 Credit Scoring and Loan Decision Models

Statistical credit scoring estimates the likelihood that a borrower will repay a loan and has been studied for many decades. Lessmann et al. [1] benchmarked a large number of classification algorithms on credit-scoring data and found that ensemble methods generally perform better than single classifiers such as plain logistic regression. This supports the choice made here to compare a linear baseline with two tree ensembles.

3.2 Logistic Regression

Logistic Regression models the log-odds of the positive class as a weighted sum of the input features [10]. It is fast, interpretable and widely used in credit scoring, which makes it a natural baseline. Because it is a linear model it cannot capture complicated interactions without manual feature design. The project uses it with max_iter = 2000 on standardised and one-hot encoded inputs.

3.3 Random Forest

Random Forest, introduced by Breiman [3], builds many decision trees on random samples of the data and random subsets of features and averages their votes. Averaging reduces the variance of single trees and handles non-linear relationships and interactions. The project uses 200 trees with a minimum of two samples per leaf.

3.4 Gradient Boosting

Gradient boosting [4] builds trees one after another, with each new tree trained to correct the errors of the previous ones. The project uses scikit-learn's HistGradientBoostingClassifier [7], a histogram-based implementation of the same idea that is inspired by LightGBM [11] and is fast on tabular data of this size. It is run for 300 iterations with a learning rate of 0.1.

3.5 Class Imbalance and SMOTE

When one class is rare, classifiers tend to favour the majority class. SMOTE (Synthetic Minority Over-sampling Technique) [5] creates new minority examples by interpolating between a real minority example and one of its nearest minority neighbours. It is implemented in the imbalanced-learn library [6]. A key rule is that SMOTE must be applied only to training data; validation and test data must keep the real class distribution. This project enforces the rule by placing SMOTE inside an imbalanced-learn pipeline, so that it runs only when the model is fitted.

3.6 Feature Engineering

Feature engineering creates new inputs from existing columns, for example ratios that express affordability or stability. Ratios are common in credit analysis because they remove the effect of scale (a loan of a given size means something different to a high-income and a low-income applicant). Tree ensembles can often learn such relations on their own, so the usefulness of hand-made features has to be tested empirically, as is done in Section 5.4.

3.7 Evaluation Metrics

For imbalanced problems, accuracy is supplemented by precision, recall and F1-score. ROC-AUC measures how well the model ranks positive applications above negative ones across all thresholds [8]. Precision–recall analysis

is often more informative than ROC when the positive class is rare [9]. This project reports accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix.

3.8 Permutation Importance

Permutation importance measures how much a model's score drops when the values of one feature are randomly shuffled. A large drop means that the model relies strongly on that feature. The idea was introduced for Random Forests by Breiman [3] and is available in scikit-learn [7]. It measures reliance of the model on a feature, not a causal effect, and it can be slightly negative for features that carry no useful information.

3.9 Literature Summary

Table 3.1. Summary of relevant literature

Area	Relevant contribution	Use in this project
Credit scoring benchmarks [1]	Compares many classifiers on credit data; ensembles tend to perform well.	Motivates comparing a linear model with tree ensembles.
Logistic Regression [10]	Interpretable probabilistic linear classifier.	Baseline model.
Random Forest [3]	Ensemble of randomised decision trees; permutation importance.	Second model and source of the importance idea.
Gradient boosting [4], [11]	Sequentially built trees; histogram-based fast variants.	Selected model (HistGradientBoosting).
SMOTE [5], [6]	Synthetic minority over-sampling by interpolation.	Balances the training set only.
ROC and PR analysis [8], [9]	Threshold-independent evaluation; value of PR for rare classes.	Metrics beyond accuracy.
Scikit-learn [7]	Pipelines, preprocessing, models, metrics.	Implementation of the whole workflow.

3.10 Research Gap and Project Positioning

The project does not propose a new algorithm. Its contribution is an applied, carefully evaluated workflow: a leakage-safe pipeline in which imputation, scaling, encoding, SMOTE and the classifier are all fitted on training data only; a transparent Credit Profile Score whose calculation is fully documented; and an honest test of whether derived features help. The report also states the limitations of the data and of the results.

CHAPTER 4

METHODOLOGY

4.1 Overall Approach

The methodology starts with loading the raw loan data and cleaning impossible values. Six features are derived from the original columns. The data is then split in a stratified manner into training, validation and test sets in the ratio 70 : 15 : 15. Preprocessing (imputation, scaling and encoding), SMOTE and the classifier are combined into one pipeline that is fitted on the training set only. Three classifiers are compared on the validation set, the best one is selected by F1-score, and a single final evaluation is made on the test set. Finally, permutation importance and an example prediction are used to explain the model.

4.2 System Architecture

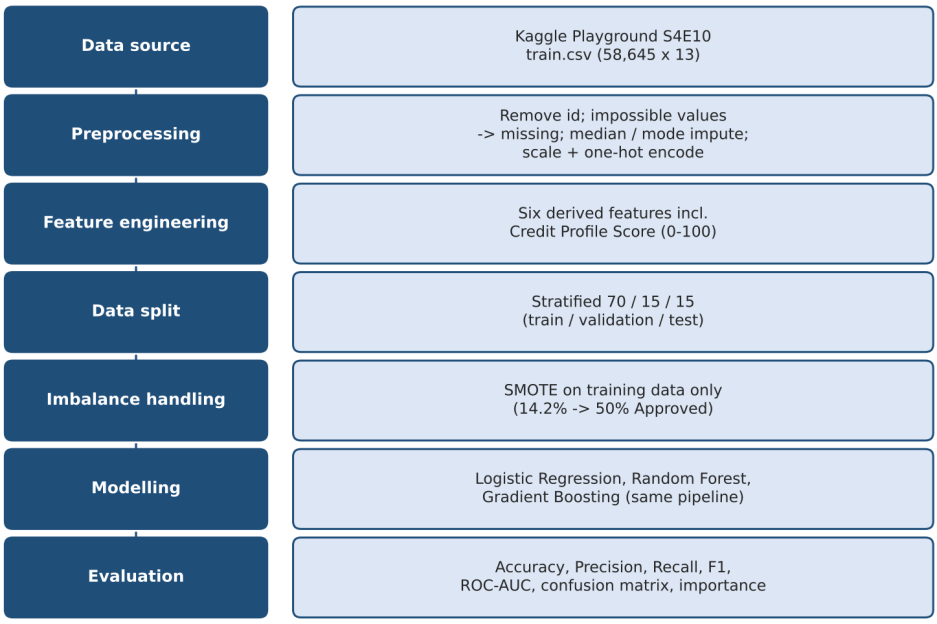


Figure 4.1. Overall system architecture (logical view). Each layer has one responsibility, and only the training set is used to fit any component.

4.3 Dataset Description

The dataset is the Kaggle Playground Series Season 4 Episode 10 loan dataset (train.csv) [2]. It has 58,645 rows and 13 columns: 11 input attributes, an identifier (id) and the target (loan_status). The raw file contains no missing values and no duplicate rows. Table 4.1 lists the attributes with an example value from the first row of the file.

Table 4.1. Dataset attributes

Attribute	Description	Example / note
id	Row identifier (removed before modelling).	0
person_age	Age of the applicant in years.	37
person_income	Annual income of the applicant.	35000

Attribute	Description	Example / note
person_home_ownership	Home-ownership status.	RENT (RENT, MORTGAGE, OWN, OTHER)
person_emp_length	Employment length in years.	0.0
loan_intent	Purpose of the loan (6 categories).	EDUCATION
loan_grade	Loan grade category, A to G.	B
loan_amnt	Requested loan amount.	6000
loan_int_rate	Interest rate of the loan (%).	11.49
loan_percent_income	Loan amount as a fraction of annual income.	0.17
cb_person_default_on_file	Previous default recorded in the credit file.	N (N or Y)
cb_person_cred_hist_length	Length of credit history in years.	14
loan_status	Target: 0 = Rejected, 1 = Approved (dataset definition).	0

The attributes fall into three groups: applicant (person_age, person_income, person_home_ownership, person_emp_length), loan (loan_intent, loan_grade, loan_amnt, loan_int_rate, loan_percent_income) and credit history (cb_person_default_on_file, cb_person_cred_hist_length). The target has 50,295 records of class 0 (85.76%) and 8,350 records of class 1 (14.24%).

4.4 Data Preprocessing

Preprocessing is intentionally simple and is performed in two stages. First, impossible values are handled directly on the data. Second, imputation, scaling and encoding are performed inside the model pipeline so that their parameters are learned from the training set only.

Table 4.2. Data preprocessing summary

Step	Action and observed result
Initial dataset	58,645 rows and 13 columns; 0 missing values and 0 duplicate rows in the raw file.
Remove identifier	id is only a row number and is excluded, leaving 11 input attributes and the target.
Fix impossible values	person_age > 100 and person_emp_length > 60 (values of 123 years) are set to missing, not deleted. After this step 3 values are missing (1 in person_age and 2 in person_emp_length).
Impute missing values	Median for numeric columns; most frequent value for categorical columns.
Scale numeric columns	StandardScaler (zero mean, unit variance).
Encode categorical columns	One-hot encoding with handle_unknown = "ignore" for person_home_ownership, loan_intent, loan_grade and cb_person_default_on_file.
Final feature set	17 input features: 11 original attributes and 6 derived features.

Other extreme values, for example the maximum income of 1,900,000, were retained because they are valid records rather than impossible values. Scaling and the tree-based models limit the influence of such values.

4.5 Feature Engineering

Six features were derived from the original columns. All of them are row-wise functions of the same application, none of them uses the target, and every division is protected so that a zero or negative denominator produces a missing value instead of an error. Missing values created in this way are imputed by the pipeline.

Table 4.3. Derived (engineered) features

No.	Derived feature	How it is calculated	Why it may matter
1	employment_stability_ratio	person_emp_length / person_age	Share of life spent working; a sign of job stability.
2	credit_maturity_ratio	cb_person_cred_hist_length / person_age	How long the applicant has had credit relative to age.
3	credit_profile_score	Weighted 0–100 score (Section 4.6)	One summary of income, affordability, employment, credit history and default.
4	loan_to_income_burden	loan_amnt / person_income	Repayment burden: loan size relative to annual income.
5	income_per_employment_year	person_income / (person_emp_length + 1)	Income relative to work experience (exploratory).
6	credit_history_coverage	cb_person_cred_hist_length / (person_emp_length + 1)	Credit history relative to work experience (exploratory).

The "+ 1" in features 5 and 6 avoids division by zero for applicants whose employment length is 0 years.

4.6 Credit Profile Score: Definition and Calculation

The Credit Profile Score is a single number between 0 and 100 that summarises how strong an applicant's financial and credit profile looks, using only information that is available in the application. A higher score means a stronger profile. It is an experimental, rule-based score designed for this project: the point weights were chosen by reasoning about credit risk and were not learned from the data, and the score is not an industry credit score such as those issued by credit bureaus.

4.6.1 Components and formulas

The score is the sum of five components. Each component grows linearly with the attribute until a cap is reached and stays at its maximum afterwards, so that extreme values (for example a very large income) cannot dominate the score. The maximum points add up to 100.

Table 4.4. Credit Profile Score components and formulas

Component	Max points	Formula	Meaning
Income	20	20 × min(person_income ÷ 100,000, 1)	Full marks at an annual income of 100,000 or more.
Loan affordability	30	30 × (1 − min(loan_percent_income ÷ 0.5, 1))	Full marks for a very small loan relative to income; zero when the loan is 50% of income or more.
Employment length	15	15 × min(person_emp_length ÷ 10, 1)	Full marks at 10 or more years of employment.
Credit-history length	20	20 × min(cb_person_cred_hist_length ÷ 15, 1)	Full marks at 15 or more years of credit history.
No previous default	15	15 if cb_person_default_on_file = N, otherwise 0	Rewards a clean credit file.
Total	**100**	Sum of the five components, clipped to the range 0–100	**Credit Profile Score**

In symbols, with P_inc, P_aff, P_emp, P_hist and P_def denoting the points of the five components:

$$\textbf{Credit Profile Score} = \text{clip}(P_{inc} + P_{aff} + P_{emp} + P_{hist} + P_{def} , 0 , 100)$$

The loan-affordability component has the largest weight (30 points) because the size of a loan relative to income is the most direct indicator of repayment pressure. The default flag is a yes/no component, so it is given a fixed block of 15 points.

4.6.2 Handling of missing inputs

If an input needed by a component is missing, that component receives half of its maximum points instead of zero, so that a missing value neither rewards nor punishes the applicant: 10 points for income, 15 for affordability, 7.5 for employment length and 10 for credit-history length. The default flag has no missing values. In the cleaned dataset only 3 values are affected, so this rule matters for very few records.

4.6.3 Worked examples

Example 1 – first record of the dataset (income 35,000; loan_percent_income 0.17; employment 0 years; credit history 14 years; no previous default):

- Income points = 20 × min(35,000 ÷ 100,000, 1) = 20 × 0.35 = 7.00
- Affordability points = 30 × (1 − min(0.17 ÷ 0.5, 1)) = 30 × 0.66 = 19.80
- Employment points = 15 × min(0 ÷ 10, 1) = 0.00
- Credit-history points = 20 × min(14 ÷ 15, 1) = 18.67
- No-default points = 15 (default flag is N)
- **Credit Profile Score = 7.00 + 19.80 + 0.00 + 18.67 + 15 = 60.47**

Example 2 – high-risk applicant (the example used in Section 5.8: income 30,000; loan_percent_income 0.50; employment 1 year; credit history 3 years; previous default):

- Income points = 20 × 0.30 = 6.00; affordability points = 30 × (1 − 1) = 0.00
- Employment points = 15 × 0.1 = 1.50; credit-history points = 20 × 0.2 = 4.00
- No-default points = 0 (default flag is Y)
- **Credit Profile Score = 6.00 + 0.00 + 1.50 + 4.00 + 0 = 11.50**

4.6.4 Resulting score distribution

Over the 58,645 applications the score ranges from 9.5 to 99.4, with a mean of 59.48, a standard deviation of 13.09, a median of 59.81 and quartiles of 50.97 and 68.38. How the score relates to the target is analysed in Section 5.5.

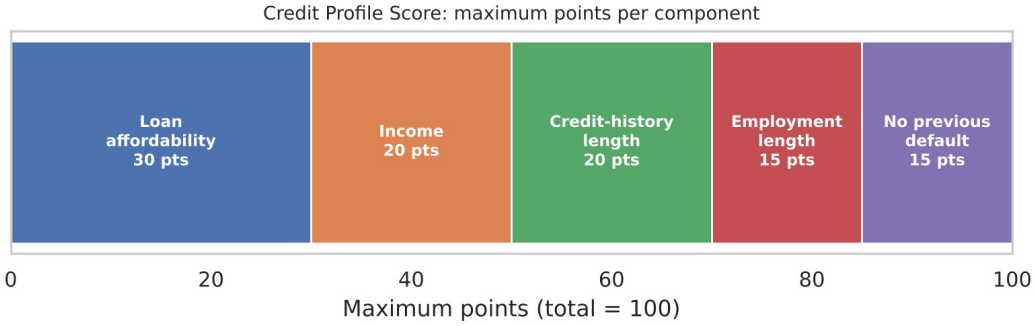


Figure 4.2. Maximum points contributed by each component of the Credit Profile Score.

4.7 Stratified Data Split and Leakage Prevention

The data was split in a stratified manner (random_state = 42) so that each part keeps the original 14.24% share of class 1. First 15% of the data was set aside as the test set; the remaining data was split again to obtain a validation set of the same size as the test set.

Table 4.5. Stratified data split

Partition	Share	Records	Purpose
Training	70%	41,051	Fit the preprocessing, SMOTE and the models.
Validation	15%	8,797	Compare models, compare feature sets and compute permutation importance.
Test	15%	8,797	Final, untouched evaluation of the selected model (used once).

Leakage is prevented in three ways. First, the imputers, scaler, encoder and SMOTE are part of one pipeline and are fitted on training data only. Second, SMOTE is never applied to the validation or test data, which keep the real class proportions. Third, the derived features use only values from the same row and never use the target.

4.8 Handling Class Imbalance with SMOTE

SMOTE creates new minority-class records by interpolating between a real class-1 record and one of its nearest class-1 neighbours in the preprocessed feature space. In the training set it increases the number of class-1 records from 5,845 to 35,206, which equals the number of class-0 records (35,206). The training set therefore grows from 41,051 to 70,412 records, of which 29,361 are synthetic, and the share of class 1 rises from 14.2% to 50%.

4.9 Models and Configurations

Three models were compared. All of them run inside the same pipeline: imputation, scaling and encoding, then SMOTE (training only), then the classifier. An application is predicted as class 1 when the predicted probability is at least 0.5.

Table 4.6. Model configurations

Model	Configuration used	Role
Logistic Regression	max_iter = 2000, random_state = 42	Simple, fast, interpretable baseline.
Random Forest	n_estimators = 200, min_samples_leaf = 2, random_state = 42, n_jobs = -1	Ensemble of trees; captures non-linear patterns.
Gradient Boosting	HistGradientBoostingClassifier, max_iter = 300, learning_rate = 0.1, random_state = 42	Trees built one after another, each correcting earlier errors.

These are fixed configurations. The project did not run a systematic hyperparameter search, so the results should not be described as the best achievable by each algorithm.

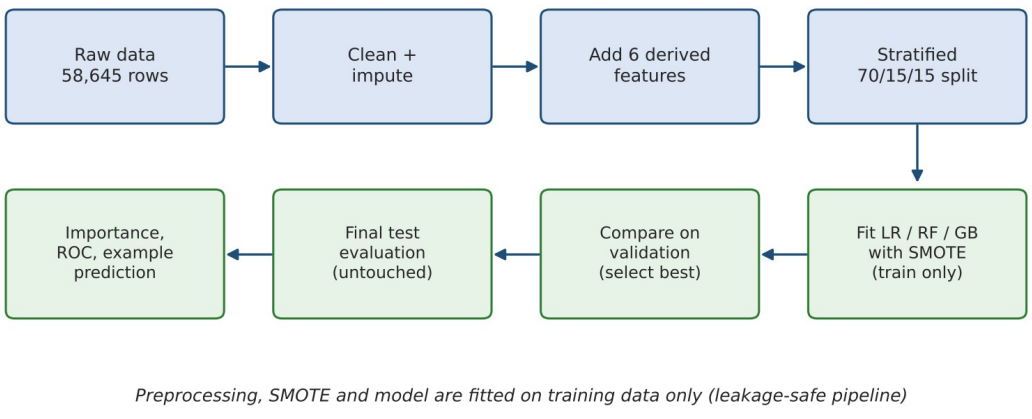


Figure 4.3. End-to-end methodology workflow.

4.10 Model Selection and Evaluation Metrics

The best model is selected on the validation set by the highest F1-score. The test set is used only once, for the selected model. Because the classes are imbalanced, several metrics are reported.

Table 4.7. Evaluation metrics

Metric	Definition	Purpose
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall fraction of correct predictions.
Precision	TP / (TP + FP)	Of the applications predicted as class 1, how many truly are class 1.
Recall	TP / (TP + FN)	Of all true class-1 applications, how many are found.
F1-score	2 × Precision × Recall / (Precision + Recall)	Balance of precision and recall; used for model selection.
ROC-AUC	Area under the ROC curve	Ranking ability across all thresholds.
Confusion matrix	Counts of TN, FP, FN and TP	Shows the type and number of errors.

Accuracy alone is not sufficient: a classifier that always predicts class 0 would obtain about 85.8% accuracy on this data while finding none of the class-1 applications.

4.11 Feature Importance Method

Permutation importance was computed with the selected pipeline on the validation set. Each raw input feature is shuffled in turn, the pipeline predicts again, and the decrease in F1-score is recorded. The procedure used 5 repeats per feature (random_state = 42). A larger mean decrease means that the model relies more on that feature. The measure describes model reliance, not causation, and values slightly below zero can occur for features that add nothing.

4.12 Tools

The work was implemented in Python using pandas and NumPy for data handling, scikit-learn for preprocessing, models, metrics and permutation importance, imbalanced-learn for SMOTE and the imbalanced-learn pipeline, and matplotlib for the figures. The random seed was fixed at 42 for all random components.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Exploratory Data Analysis

The target is strongly imbalanced: 50,295 applications (85.76%) belong to class 0 and 8,350 (14.24%) to class 1, an imbalance ratio of about 6 : 1. Figure 5.1 shows the real class distribution before any resampling.

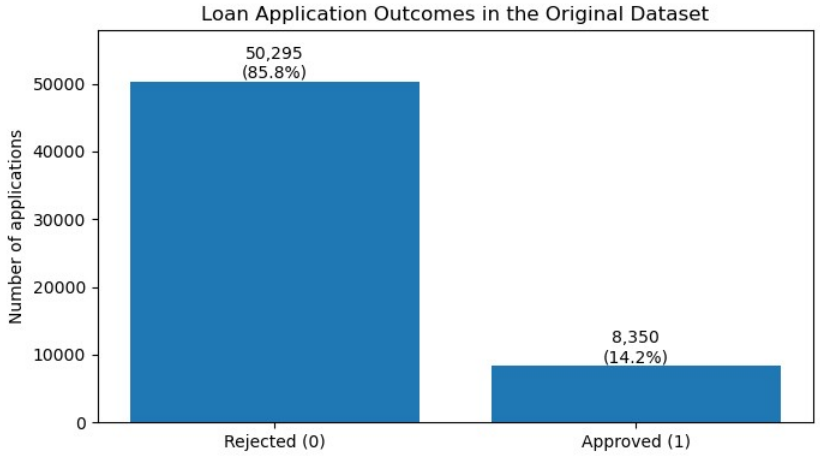


Figure 5.1. Class distribution in the original dataset.

Table 5.1 summarises the numerical attributes after the impossible values were set to missing. Applicants are young (median age 26, mean 27.55 years) and the income distribution is strongly right-skewed (median 58,000 against a maximum of 1,900,000). The median loan is 8,000 at an interest rate of about 10.75%, and a typical loan equals 14% of annual income.

Table 5.1. Summary statistics of numerical attributes (after cleaning)

Attribute	Mean	Std	Min	25%	Median	75%	Max
person_age	27.55	6.02	20	23	26	30	84
person_income	64,046.17	37,931.11	4,200	42,000	58,000	75,600	1,900,000
person_emp_length	4.70	3.90	0	2	4	7	41
loan_amnt	9,217.56	5,563.81	500	5,000	8,000	12,000	35,000
loan_int_rate	10.68	3.03	5.42	7.88	10.75	12.99	23.22
loan_percent_income	0.16	0.09	0.00	0.09	0.14	0.21	0.83
cb_person_cred_hist_length	5.81	4.03	2	3	4	8	30

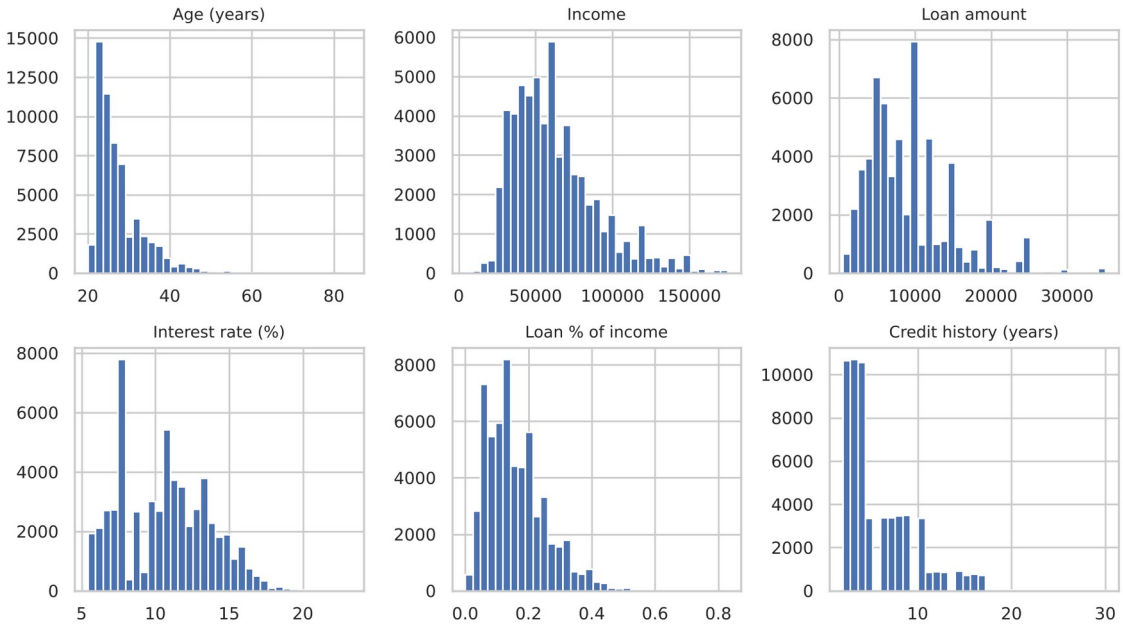


Figure 5.2. Distributions of the numerical input attributes (income is shown up to its 99th percentile).

Among the categorical attributes, renters (30,594 applications, 52.2%) and mortgage holders (24,824, 42.3%) dominate, while only 3,138 applicants own their home and 89 are in the OTHER group. Education (12,271) and medical (10,934) are the most frequent loan purposes. Loan grades A and B together cover about 71% of the data, whereas grades F and G are very rare (149 and 33 records). A previous default is recorded for 8,702 applicants (14.8%).

Figure 5.3 shows the share of class-1 records within each category. The share rises sharply with the loan grade (4.9% for grade A, 10.2% for B, 13.5% for C and between 59% and 82% for grades D to G; grade G has only 33 records). It is also higher for applicants with a previous default (29.9% against 11.5%) and for renters (22.3%) compared with mortgage holders (6.0%) and owners (1.4%). Debt consolidation (18.9%), medical (17.8%) and home improvement (17.4%) loans have higher class-1 shares than venture (9.3%) and education (10.8%) loans.

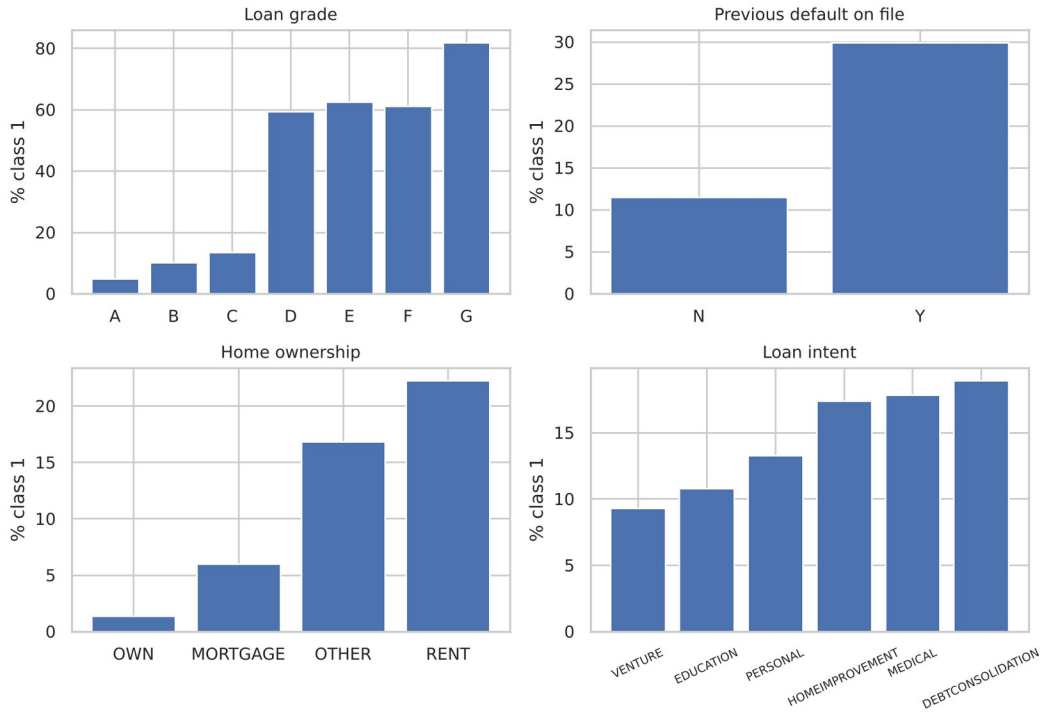


Figure 5.3. Share of class 1 (loan_status = 1) by categorical input attributes.

The Spearman correlation heatmap (Figure 5.4) shows that the target is positively associated with loan_int_rate (0.32) and loan_percent_income (0.29), and negatively with person_income (−0.24). loan_amnt (0.13) and person_emp_length (−0.12) show weaker association, while person_age and cb_person_cred_hist_length are almost uncorrelated with the target (about −0.01). Correlation indicates association, not causation.

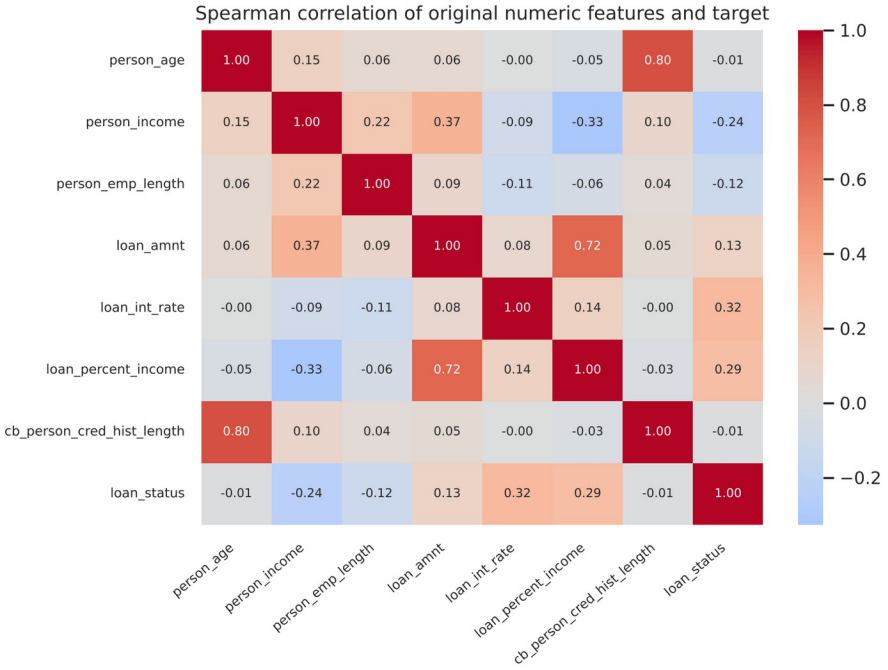


Figure 5.4. Spearman correlation of the original numeric features and the target.

5.2 Effect of SMOTE on the Training Set

Before SMOTE the training set contained 35,206 records of class 0 and 5,845 of class 1 (14.2%). After SMOTE both classes contain 35,206 records, so the training set grows from 41,051 to 70,412 records and class 1 makes up 50% of it (Figure 5.5). The 29,361 additional class-1 records are synthetic. The validation and test sets are not resampled and keep the real class proportions, so their metrics reflect realistic conditions.

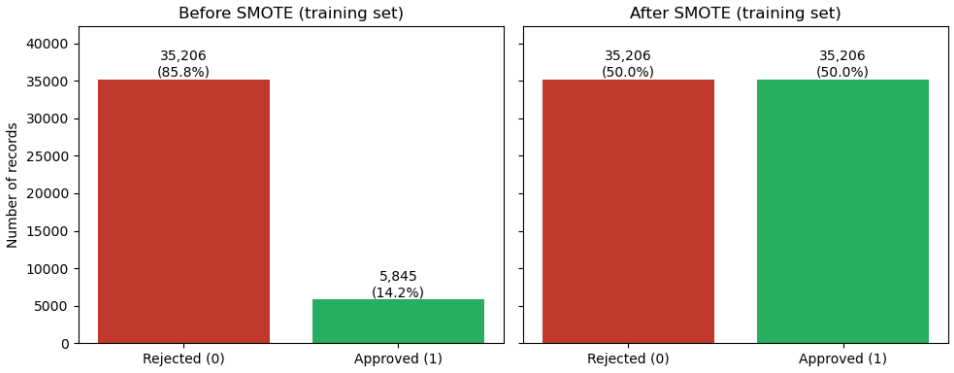


Figure 5.5. Training set before and after SMOTE.

5.3 Validation Model Comparison

Table 5.2 and Figure 5.6 compare the three models on the validation set at the threshold of 0.5. Gradient Boosting is the best model overall, with the highest accuracy (0.9486), precision (0.9008), F1-score (0.7991) and ROC-AUC (0.9519). Logistic Regression has the highest recall (0.8107) but a very low precision (0.4752): it finds more class-1 applications but also flags many class-0 applications wrongly, which lowers its accuracy to 0.8456. Random Forest is a close second (F1-score 0.7872, ROC-AUC 0.9284). The validation F1-score was used to choose Gradient Boosting as the final model.

Table 5.2. Validation model comparison (threshold 0.5)

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.8456	0.4752	0.8107	0.5992	0.9007
Random Forest	0.9455	0.8869	0.7077	0.7872	0.9284
Gradient Boosting	**0.9486**	**0.9008**	0.7181	**0.7991**	**0.9519**

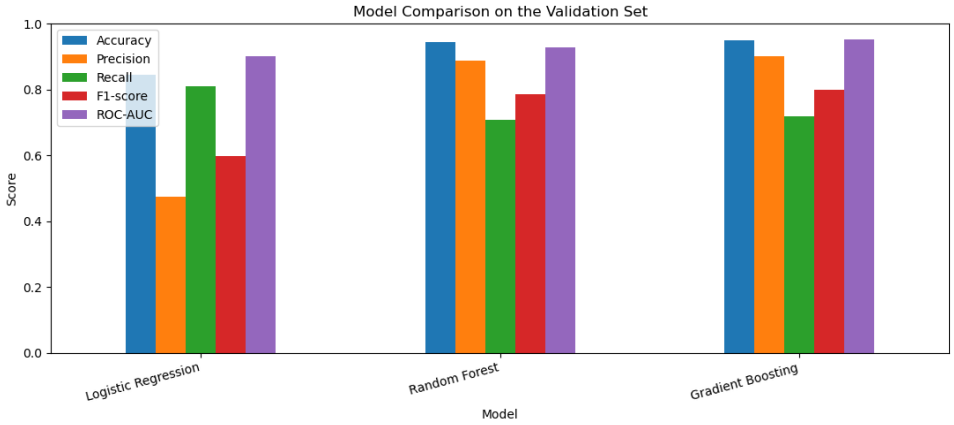


Figure 5.6. Model comparison on the validation set.

5.4 Original versus Derived Features

To test whether the six derived features are useful, each model was trained twice with the same pipeline and settings: once with the 11 original features and once with the original features plus the six derived features. Both versions were evaluated on the validation set (Table 5.3, Figure 5.7); the test set was not used.

Table 5.3. Original versus derived features (validation set)

Model	Feature set	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	Original	0.4705	0.8083	0.5948	0.8995
Logistic Regression	Original + 6 derived	0.4752	0.8107	0.5992	0.9007
Random Forest	Original	0.8663	0.7141	0.7828	0.9340
Random Forest	Original + 6 derived	0.8869	0.7077	0.7872	0.9284
Gradient Boosting	Original	0.9026	0.7252	0.8043	0.9481
Gradient Boosting	Original + 6 derived	0.9008	0.7181	0.7991	0.9519

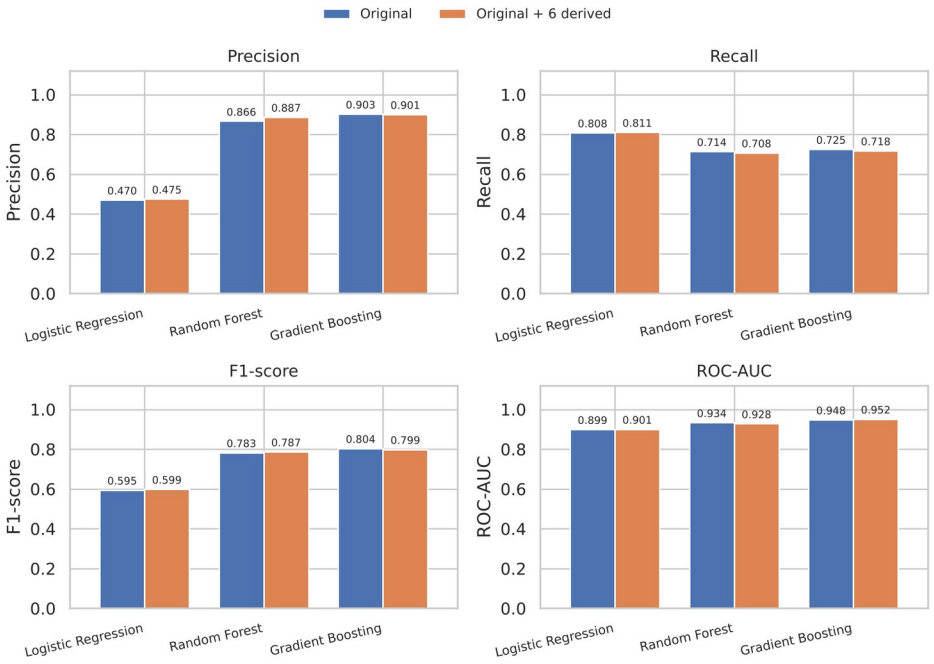


Figure 5.7. Original features versus original + six derived features on the validation set.

The effect of the derived features is small and mixed. F1-score changes by +0.0044 for Logistic Regression, +0.0044 for Random Forest and −0.0052 for Gradient Boosting; ROC-AUC changes by +0.0012, −0.0056 and +0.0038 respectively. Because the changes are of both signs and no significance test or repeated-split evaluation was performed, they should be read as “no clear benefit” and not as a proven gain or loss. The original features already carry most of the signal, and tree ensembles can learn ratios of existing columns by themselves. The final pipeline of this project is the version that includes the six derived features. For Gradient Boosting, the original-feature version has a slightly higher validation F1-score (0.8043 against 0.7991), while the derived-feature version has a slightly higher ROC-AUC (0.9519 against 0.9481); both differences are small.

5.5 Credit Profile Score Analysis

This section examines how the Credit Profile Score defined in Section 4.6 behaves on the data. The score is computed for all 58,645 applications. Its mean is 59.48 (standard deviation 13.09) and its range is 9.5 to 99.4.

5.5.1 Relationship with the target

Applications of class 1 have a clearly lower score than applications of class 0: the mean score is 47.95 for class 1 against 61.39 for class 0, and the medians are 47.20 and 61.40 (Figure 5.8). The Pearson correlation between the score and loan_status is −0.359, compared with +0.378 for loan_percent_income, the strongest single raw predictor in the importance analysis. The score therefore ranks applicants in a meaningful way.

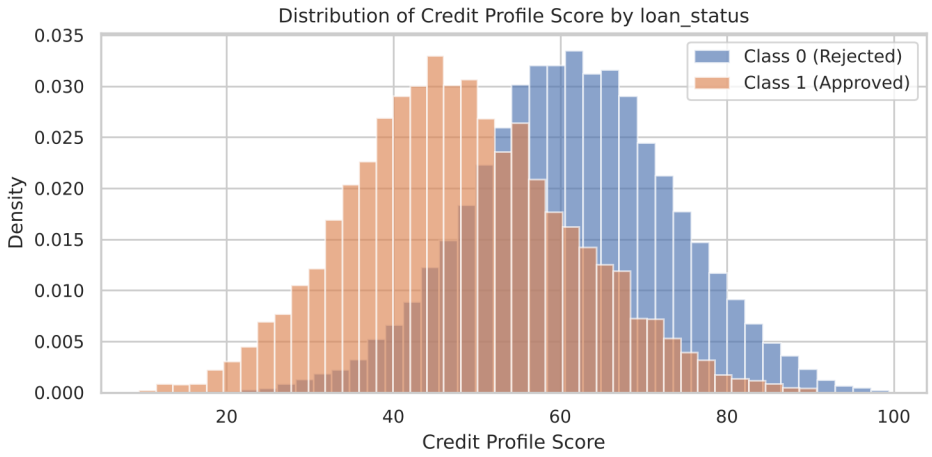


Figure 5.8. Distribution of the Credit Profile Score for class 0 and class 1.

The same pattern is seen when the score is grouped into bands (Figure 5.9). The share of class-1 records falls steadily as the score rises: 54.2% for scores below 40 (4,303 applications), 28.1% for 40–49 (9,039), 12.0% for 50–59 (16,284), 6.4% for 60–69 (16,627), 4.1% for 70–79 (9,141) and 2.5% for scores of 80 and above (3,251).

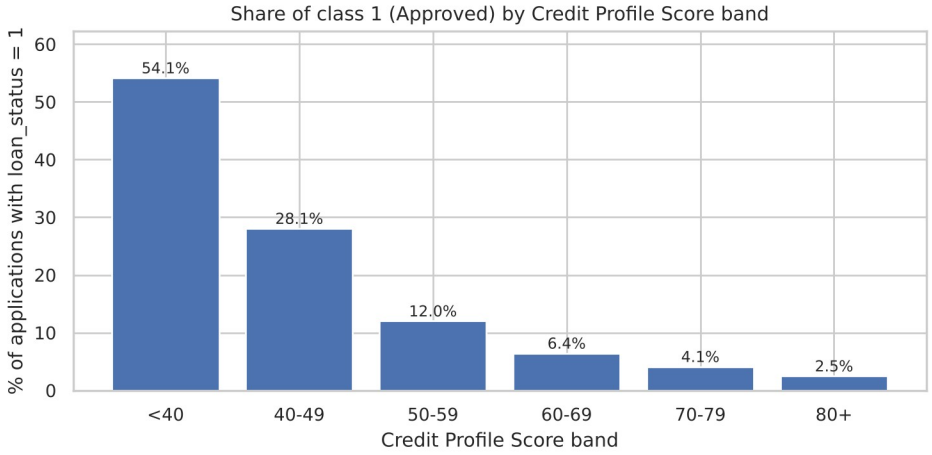


Figure 5.9. Share of class 1 by Credit Profile Score band.

5.5.2 Contribution of each component

Table 5.4 shows the average points earned by each class in each component. Loan affordability (21.29 against 15.34 points) and income (12.49 against 9.50) separate the two classes most, followed by the default component (13.18 against 10.33) and employment length (6.81 against 5.24). The credit-history component hardly differs between the classes (7.62 against 7.53 points), in line with the near-zero correlation between cb_person_cred_hist_length and the target.

Table 5.4. Credit Profile Score statistics by class (mean points)

Component (max points)	Class 0	Class 1
Income (20)	12.49	9.50
Loan affordability (30)	21.29	15.34
Employment length (15)	6.81	5.24
Credit-history length (20)	7.62	7.53
No previous default (15)	13.18	10.33
Credit Profile Score (100): mean	**61.39**	**47.95**
Credit Profile Score: median	61.40	47.20
Credit Profile Score: standard deviation	12.06	13.09

5.5.3 Value as a model feature

Although the score is informative on its own, it did not help the models. In the permutation-importance analysis (Section 5.6) its importance is −0.0009 (about zero), and in the feature comparison of Section 5.4 the derived feature set did not give a clear improvement. The reason is that the score is a deterministic combination of columns that the models already receive (income, loan_percent_income, employment length, credit-history length and default flag), so it adds no new information; the models can learn these relations, and more, directly from the raw columns. The score is best seen as an easy-to-explain summary of an applicant's profile, not as a replacement for the model.

5.5.4 Observation about the direction of the label

Section 5.5.1 shows that the applications with class 1 have lower scores, that is, weaker profiles. Section 5.1 shows the same pattern: class 1 is more common for low loan grades, previous defaults, renters and high loan burdens. If class 1 really meant "approved", a lender would be approving weaker profiles more often than stronger ones, which is unusual. This pattern is typical of a default-risk label. The project follows the dataset description (1 = Approved), but the meaning of the label should be confirmed with the data source before the model is used for any real decision (see Section 5.10).

5.6 Feature Importance

Permutation importance was computed for the selected Gradient Boosting pipeline on the validation set, using the decrease in F1-score (Table 5.5, Figure 5.10). The five most influential features are loan_percent_income (0.1702), person_home_ownership (0.1639), loan_intent (0.1470), loan_grade (0.1253) and loan_int_rate (0.0709). They are followed by person_age (0.0452), person_income (0.0398) and person_emp_length (0.0279).

Among the derived features only loan_to_income_burden has a visible effect (0.0308, rank 8 of 17). credit_maturity_ratio (0.0026) and income_per_employment_year (0.0008) are close to zero, and credit_history_coverage, credit_profile_score and employment_stability_ratio are slightly negative, meaning that shuffling them does not hurt the model. Importance of correlated features can be shared or hidden: for example, loan_to_income_burden and loan_percent_income describe similar information. The values describe how much the model relies on a feature, not whether the feature causes the outcome.

Table 5.5. Permutation feature importance (decrease in validation F1-score)

Rank	Feature	Mean F1 decrease	Std
1	loan_percent_income	0.17020	0.00629
2	person_home_ownership	0.16386	0.00915
3	loan_intent	0.14699	0.00436
4	loan_grade	0.12531	0.00429
5	loan_int_rate	0.07091	0.00564
6	person_age	0.04517	0.00330
7	person_income	0.03976	0.00375
8	loan_to_income_burden (derived)	0.03081	0.00330
9	person_emp_length	0.02792	0.00272
10	credit_maturity_ratio (derived)	0.00255	0.00148
11	loan_amnt	0.00171	0.00220
12	income_per_employment_year (derived)	0.00082	0.00181
13	cb_person_default_on_file	0.00025	0.00040
14	credit_history_coverage (derived)	−0.00022	0.00079
15	credit_profile_score (derived)	−0.00089	0.00184
16	cb_person_cred_hist_length	−0.00096	0.00122
17	employment_stability_ratio (derived)	−0.00153	0.00155

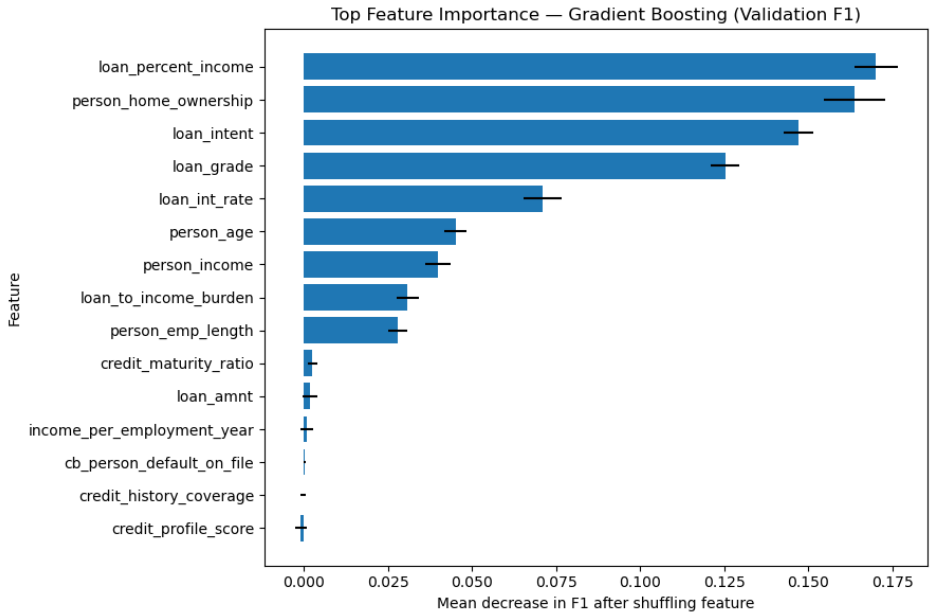


Figure 5.10. Top 15 features by permutation importance (Gradient Boosting, validation F1-score).

5.7 Final Test Performance

The selected Gradient Boosting pipeline was evaluated once on the untouched test set of 8,797 applications (7,544 of class 0 and 1,253 of class 1). Table 5.6 gives the metrics. The results are close to the validation results (F1-score 0.7991 on validation and 0.8144 on test; ROC-AUC 0.9519 and 0.9531), which indicates that the model generalises and was not over-fitted to the validation set.

Table 5.6. Final test performance (Gradient Boosting, threshold 0.5)

Metric	Test value	Interpretation
Accuracy	0.9525	About 95.25% of predictions are correct.
Precision	0.9179	About 91.8% of the applications predicted as class 1 truly are class 1.
Recall	0.7318	About 73.2% of the true class-1 applications are found.
F1-score	0.8144	Balance of precision and recall.
ROC-AUC	0.9531	Very good ranking ability of class 1 above class 0.

The confusion matrix (Figure 5.11) shows 7,462 true negatives, 82 false positives, 336 false negatives and 917 true positives. Only 82 of the 7,544 class-0 applications (about 1.1%) are wrongly predicted as class 1, but 336 of the 1,253 class-1 applications (about 26.8%) are missed. Precision is therefore much higher than recall: positive predictions are reliable, but the model does not find every class-1 application. The ROC curves on the test set give AUC values of 0.953 for Gradient Boosting, 0.930 for Random Forest and 0.908 for Logistic Regression, so Gradient Boosting stays above the other two curves.

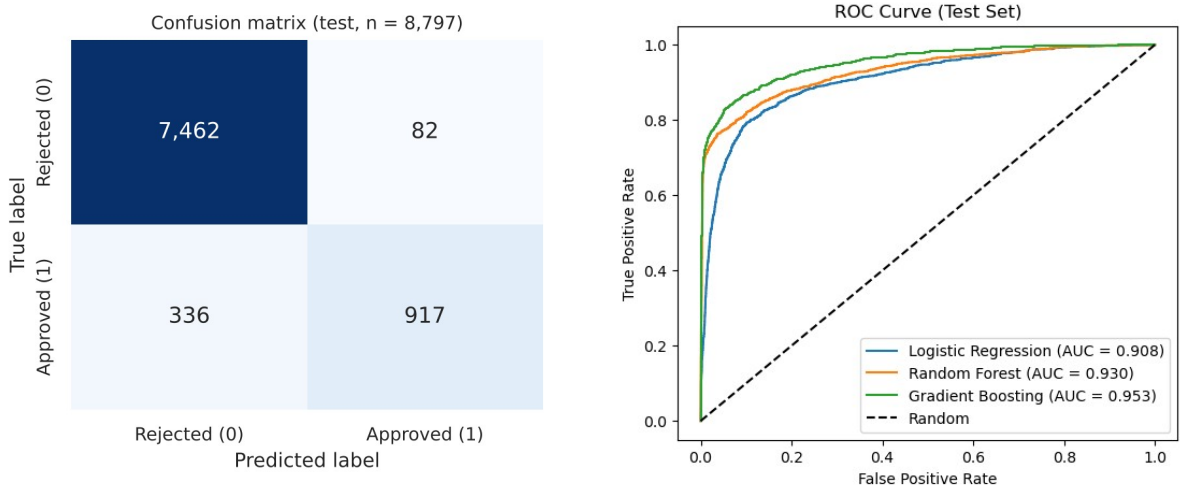


Figure 5.11. Test confusion matrix (left) and ROC curves of the three models on the test set (right).

Table 5.7. Test classification report (Gradient Boosting)

Class	Precision	Recall	F1-score	Support
Rejected (0)	0.96	0.99	0.97	7,544
Approved (1)	0.92	0.73	0.81	1,253
Accuracy			0.95	8,797
Macro average	0.94	0.86	0.89	8,797
Weighted average	0.95	0.95	0.95	8,797

5.8 Example Prediction

To demonstrate the pipeline, a single new application was passed through the same derived-feature function and the selected pipeline: age 25, income 30,000, home ownership RENT, employment length 1 year, loan intent MEDICAL, loan grade E, loan amount 15,000, interest rate 17.5%, loan_percent_income 0.50, previous default Y and credit-history length 3 years. Its Credit Profile Score is 11.50 (Section 4.6.3). The model returned a probability of 0.9989 for class 1, so the prediction is class 1 (“Approved” in the dataset’s terms).

This is a very weak profile: the loan equals half of the income, the grade is low and a previous default exists. That it receives a probability of almost 1 for class 1 is consistent with the data pattern reported in Section 5.5.4, where

class 1 is concentrated among high-burden, low-grade and previous-default applications. It is a further reason to confirm the meaning of the label before real-world use.

5.9 Overall Discussion

The results show that the loan attributes contain strong information about the target. Gradient Boosting separates the two classes very well (ROC-AUC 0.953) and keeps precision above 0.91 on the test set. SMOTE balanced the training data without contaminating the evaluation data, and all preprocessing steps were fitted on training data only.

The Logistic Regression baseline shows the trade-off that exists in imbalanced problems: it achieves the highest recall (0.8107) but its precision of 0.4752 means that more than half of its positive predictions are wrong. The tree ensembles keep precision high and give up some recall. Which of the two is preferable depends on the cost of each error type, which this project does not model.

The derived features, including the Credit Profile Score, did not give a clear improvement. The Credit Profile Score is nevertheless useful as an interpretable summary: it ranks applicants in a way that agrees with the target, and its calculation can be explained component by component.

5.10 Limitations

- **Synthetic data:** the Kaggle Playground data is generated from a real data set, so real-world results may differ.
- **Meaning of the label:** the dataset defines class 1 as Approved, but class 1 is more frequent among weaker profiles (low grades, previous default, high loan burden, low Credit Profile Score). The label meaning should be verified with the data source; the results of this report hold for the dataset's loan_status as given.
- **SMOTE:** the synthetic records may not be fully realistic, although they are used only in training.
- **Recall of 73%:** 336 of the 1,253 class-1 test applications (26.8%) are missed at the threshold of 0.5.
- **Fixed threshold and settings:** the threshold was not tuned for any business cost, and the models use fixed settings without a systematic hyperparameter search.
- **Single split:** all results come from one stratified split; no cross-validation, confidence intervals or significance tests were used, so small differences (such as those in Table 5.3) are not conclusive.
- **Credit Profile Score:** the weights are design choices based on reasoning, not learned or validated against an external credit standard, and the derived features added little.
- **Fairness:** attributes such as age and home ownership are used as inputs and no fairness analysis was carried out.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This mini-project developed a complete, leakage-safe machine learning workflow for loan approval prediction on the Kaggle Playground Series S4E10 dataset of 58,645 applications. The workflow cleaned impossible values, derived six new features, split the data in a stratified 70/15/15 manner, balanced the training set with SMOTE (14.2% to 50% class 1), and compared Logistic Regression, Random Forest and Gradient Boosting.

Gradient Boosting was the best model on both the validation and the test data. On the untouched test set it achieved an accuracy of 0.9525, precision of 0.9179, recall of 0.7318, F1-score of 0.8144 and ROC-AUC of 0.9531. It found 917 of the 1,253 class-1 applications and wrongly flagged only 82 of the 7,544 class-0 applications, but it missed 336 class-1 applications, so recall remains the weak point.

The six derived features gave only a marginal and mixed change in validation performance, which shows that the original features already carry most of the signal. The Credit Profile Score, a 0–100 score built from income (20 points), loan affordability (30), employment length (15), credit-history length (20) and previous default (15), is easy to explain and ranks applicants in agreement with the target (mean 61.39 for class 0 against 47.95 for class 1), but it adds no information beyond the columns it is built from. Permutation importance shows that loan_percent_income, person_home_ownership, loan_intent, loan_grade and loan_int_rate drive the model.

The report also records the limitations honestly: the data is synthetic, the meaning of the target label should be confirmed because class 1 is concentrated among weaker profiles, the threshold and model settings were not tuned, and all conclusions come from a single split.

6.2 Future Scope

- Confirm the meaning of loan_status with the data source and, if necessary, redefine the positive class and re-run the analysis.
- Tune the decision threshold according to business costs, for example to raise recall when missing a case is expensive.
- Try other boosting models such as XGBoost and LightGBM, and other balancing methods such as ADASYN and class weighting.
- Run a systematic hyperparameter search with cross-validation, and report confidence intervals and significance tests for model and feature-set comparisons.
- Add SHAP explanations and check probability calibration.
- Carry out a fairness analysis for attributes such as age and home ownership.
- Learn or validate the weights of the Credit Profile Score instead of fixing them by hand.
- Show applicants what they could improve to increase their chances of approval.
- Develop a web application in which users enter their details and obtain a loan prediction.

REFERENCES

[1] S. Lessmann, B. Baesens, H.-V. Seow and L. C. Thomas, “Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research,” European Journal of Operational Research, vol. 247, no. 1, pp. 124–136, 2015.

[2] W. Reade and A. Chow, “Binary Classification with a Loan Approval Dataset,” Kaggle Playground Series – Season 4, Episode 10, 2024.

[3] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001.

[4] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[6] G. Lemaître, F. Nogueira and C. K. Aridas, “Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning,” Journal of Machine Learning Research, vol. 18, no. 17, pp. 1–5, 2017.

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] T. Fawcett, “An introduction to ROC analysis,” Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006.

[9] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” PLOS ONE, vol. 10, no. 3, e0118432, 2015.

[10] D. W. Hosmer, S. Lemeshow and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ: Wiley, 2013.

[11] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems, vol. 30, 2017.